

INTRODUCTION

Cryptography is the study and practice of techniques used to protect information and communication from unauthorized access or manipulation. In a digital network, information may travel through systems that are not completely under the control of the sender or receiver. A central problem therefore arises: how can two people communicate securely when other parties may be able to observe the communication?

Cryptographic systems address this problem by transforming readable information into a protected form and by using carefully designed mathematical procedures for encryption, decryption, authentication, integrity and key establishment. The source project describes this process using the sequence Plaintext → Encryption → Ciphertext → Decryption and emphasizes a key principle: cryptography is built on mathematical problems that can be easy to perform in one direction but computationally difficult to reverse without special information.

Discrete mathematics provides much of the language and structure needed for these systems. Modular arithmetic allows calculations to be performed within a defined numerical system. Number theory supplies prime numbers, divisibility, factorization and Euler's totient function. The Euclidean algorithm and modular inverses support key-generation operations. Group theory and finite fields provide structured algebraic environments used by key-exchange, elliptic-curve and symmetric-key constructions. Combinatorics, probability and computational complexity help us reason about search spaces, collisions and the practical difficulty of attacks.

This report follows that mathematical journey and uses RSA as the main worked example. It then connects the ideas to Diffie–Hellman, finite fields, AES, hashing, practical security and the effect of quantum computing on classical cryptographic assumptions.

Discrete Mathematics → Mathematical Structures → Cryptographic Algorithms → Computational Hardness → Security → Trusted Digital Communication

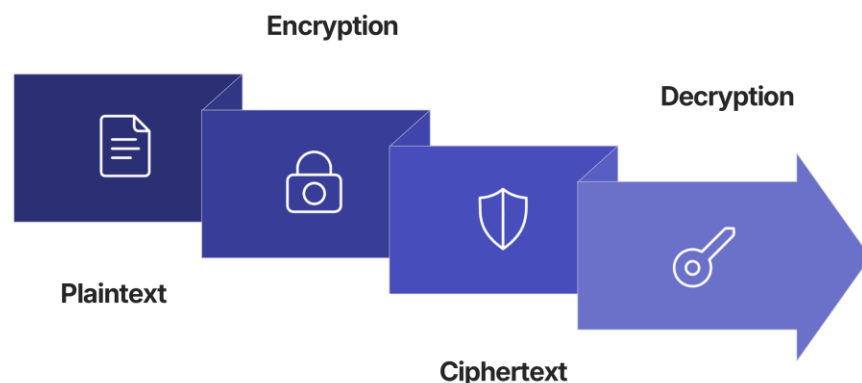


Figure 1. Plaintext–encryption–ciphertext–decryption flow reproduced from the supplied project material.

1. INTRODUCTION TO CRYPTOGRAPHY

Purpose and Context

Cryptography is the study and application of mathematical techniques used to protect information from unauthorized access, modification, or misuse. In modern digital communication, enormous amounts of information are transmitted through networks that may not be completely trusted. Personal messages, passwords, financial transactions, academic records, business documents, and authentication credentials may all need protection while they are being stored or transmitted.

At its core, cryptography addresses a fundamental communication problem: How can two parties exchange information securely when an unauthorized third party may be able to observe, intercept, or manipulate the communication channel? The solution involves mathematical transformations that make information difficult to understand without the appropriate cryptographic key or that allow the receiver to verify that the information has not been altered.

The source presentation introduces this idea through the basic process of secure communication. A sender begins with readable information, called plaintext, and applies an encryption process to produce ciphertext. The ciphertext can then be transmitted through an insecure channel. An authorized receiver uses the appropriate key and decryption procedure to recover the original plaintext.

Cryptography therefore involves much more than simply “hiding” a message. Modern cryptographic systems can support several security objectives, including confidentiality, data integrity, authentication, source authentication, and non-repudiation.

The importance of cryptography has increased with the growth of the Internet, cloud computing, online banking, e-commerce, mobile communication, and digital services. Cryptographic techniques are used to protect information both during transmission and while it is stored. NIST describes cryptography as a branch of mathematics based on transformations of data and identifies encryption as an important mechanism for protecting confidentiality.

For this project, the most important observation is that cryptography is deeply connected with discrete mathematics. The security mechanisms used in cryptographic algorithms are based on mathematical concepts such as:

- Modular arithmetic
- Prime numbers
- Greatest Common Divisor (GCD)
- Euclidean and Extended Euclidean algorithms
- Modular inverses
- Euler's Totient Function
- Exponentiation and congruences

- Sets, groups and cyclic groups
- Finite fields
- Boolean algebra
- Combinatorics and probability
- Computational complexity

These concepts provide the mathematical foundation on which many cryptographic algorithms are constructed. For example, the RSA algorithm uses prime numbers, modular arithmetic, Euler's totient function, modular inverses, and the difficulty of certain number-theoretic problems. Similarly, modern symmetric cryptography makes extensive use of operations involving finite sets, Boolean functions, and algebraic structures.

Thus, studying cryptography provides a practical demonstration of why discrete mathematics is important in computer science. Abstract mathematical ideas that may initially appear theoretical become useful when designing systems for secure digital communication.

Security Requirements in Cryptography

A secure communication system generally has to address several different requirements. These requirements are related but are not identical.

1. Confidentiality

Confidentiality means preventing unauthorized parties from accessing or understanding protected information. Encryption is commonly used to achieve this objective by transforming readable plaintext into ciphertext. Only an authorized party possessing the appropriate key should be able to recover the original information.

For example:

Plaintext $\rightarrow$ Encryption + Key $\rightarrow$ Ciphertext

The ciphertext may travel through an insecure network, but an observer should not be able to practically recover the original plaintext without the required cryptographic information.

2. Data Integrity

Confidentiality alone is not sufficient. An attacker might be unable to read a message but could still attempt to modify it.

Data integrity ensures that information has not been altered in an unauthorized manner. Cryptographic mechanisms such as message authentication codes and digital signatures can be used to detect unauthorized modifications.

For example, if a transaction originally contains:

Transfer ₹1,000

an attacker should not be able to secretly change it to:

Transfer ₹10,000

without the alteration being detected.

3. Authentication

Authentication is concerned with establishing the identity of an entity or verifying the source of information. Cryptographic mechanisms can help determine whether a message was actually generated by the claimed sender. Digital signatures and message authentication codes are examples of mechanisms that can provide authentication-related services.

4. Non-Repudiation

Certain cryptographic mechanisms, particularly digital signatures, can also support non-repudiation, where evidence is provided that can help establish the origin of a digitally signed message or transaction.

Therefore, cryptography is better understood as a collection of mathematical techniques for achieving multiple security objectives rather than merely a method for converting readable text into unreadable text.

Plaintext, Ciphertext and Keys

Three fundamental terms are used to describe the basic encryption process: plaintext, ciphertext, and key.

Plaintext

Plaintext is the original information in a readable or usable form before encryption. It may be a text message, numerical value, file, password, image, or other digital data.

For example:

Plaintext:

HELLO

Ciphertext

Ciphertext is the transformed form of the plaintext produced by an encryption algorithm. It is intended to conceal the original meaning of the information from unauthorized parties.

Conceptually:

$$\text{Plaintext} \xrightarrow{\text{Encryption}} \text{Ciphertext}$$

NIST defines encryption as a cryptographic transformation that converts plaintext into ciphertext. The corresponding reversal process is decryption.

Key

A cryptographic key is information used by a cryptographic algorithm to control its operation. The key is an essential part of the transformation and, depending on the cryptographic system, may be secret or may be distributed publicly.

A simplified representation is:

$$C = E_K(P)$$

where:

- P = Plaintext
- K = Cryptographic key
- E = Encryption algorithm
- C = Ciphertext

The receiver then applies the appropriate decryption operation:

$$P = D_K(C)$$

where D represents the decryption process.

The exact relationship between the encryption key and decryption key depends on the type of cryptographic system being used.

Symmetric and Public-Key Cryptography

Cryptographic systems can broadly be divided into symmetric-key and public-key (asymmetric-key) approaches.

Symmetric-Key Cryptography

In symmetric cryptography, the same secret key, or appropriately related secret keying material, is used for encryption and decryption.

The basic structure is:

$$P \xrightarrow{K} C \xrightarrow{K} P$$

The major requirement is that the communicating parties must have access to the secret key while preventing unauthorized parties from obtaining it.

Symmetric cryptography is particularly useful for efficiently protecting large quantities of data. Examples include modern block-cipher systems such as AES.

Public-Key Cryptography

Public-key cryptography uses a pair of mathematically related keys:

- Public key — can be distributed openly.
- Private key — must be kept secret.

This creates an important distinction from symmetric cryptography. A person can publish their public key while retaining the corresponding private key.

The RSA algorithm discussed in this project is an important example of public-key cryptography. Its construction demonstrates how discrete mathematical concepts can be combined to create a system in which certain operations are easy to perform while reversing them without the required private information is computationally difficult.

Why Discrete Mathematics Is Important?

The central theme of this project is the relationship between discrete mathematics and cryptography.

Many cryptographic operations work with integers, finite sets, logical values, and mathematical structures rather than continuous quantities. This makes discrete mathematics particularly suitable for describing and analyzing cryptographic systems.

For example, consider modular arithmetic:

$$17 \equiv 5 \pmod{12}$$

This means that 17 and 5 have the same remainder when divided by 12.

Modular arithmetic becomes extremely important when performing cryptographic calculations because it allows very large numerical operations to be represented within a finite mathematical system.

Similarly, prime numbers play an important role in RSA. If two prime numbers p and q are multiplied,

$$n = pq$$

computing n from p and q is straightforward. However, recovering the original prime factors from a sufficiently large n is computationally difficult using known classical methods. RSA uses this asymmetry as part of its mathematical foundation.

The project therefore does not treat mathematical concepts as isolated topics. Instead, it shows how they interact to form practical cryptographic algorithms.

Cryptography as an Application of Mathematics

One of the most important lessons from cryptography is that mathematical theory can have direct practical applications.

A concept such as the Extended Euclidean Algorithm may initially appear to be simply a method for finding the GCD of two integers. In cryptography, however, it can be used to determine a modular inverse, which is an important component of RSA key generation.

Similarly: $\gcd(a, m) = 1$

is the condition required for a to possess a multiplicative inverse modulo m .

Euler's Totient Function is another example. For a positive integer n , $\phi(n)$ counts the positive integers up to n that are relatively prime to n . For two distinct primes p and q ,

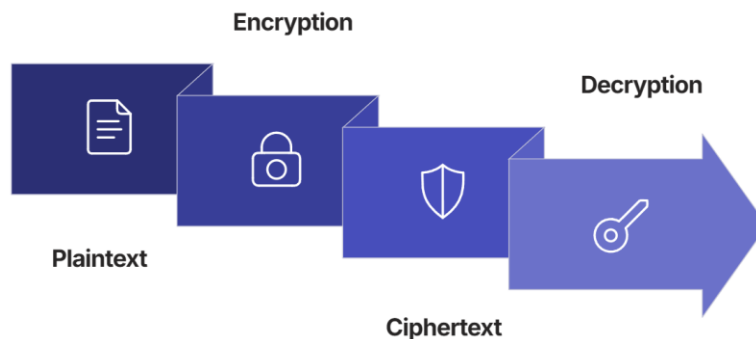
$$\phi(pq) = (p - 1)(q - 1)$$

This relationship becomes important in the construction of RSA keys.

Therefore, the progression of the project can be viewed as:

Discrete Mathematics → Mathematical Operations → Cryptographic Algorithms → Secure Communication
--

This connection between theory and application is the main focus of the project.



2. THE DISCRETE MATHEMATICS TOOLKIT

Mathematical areas used in cryptography

The source project's visual map identifies number theory, prime numbers, modular arithmetic, GCD and the Euclidean algorithm, modular inverses, group theory, finite fields, Boolean algebra, combinatorics, probability and computational complexity as major parts of the cryptographic toolkit.

Each topic contributes a different kind of structure. Number theory provides properties of integers and primes; modular arithmetic provides controlled arithmetic; algebraic structures provide rules for operations; probability helps quantify collisions and uncertainty; and computational complexity helps distinguish a theoretical attack from a practical one.

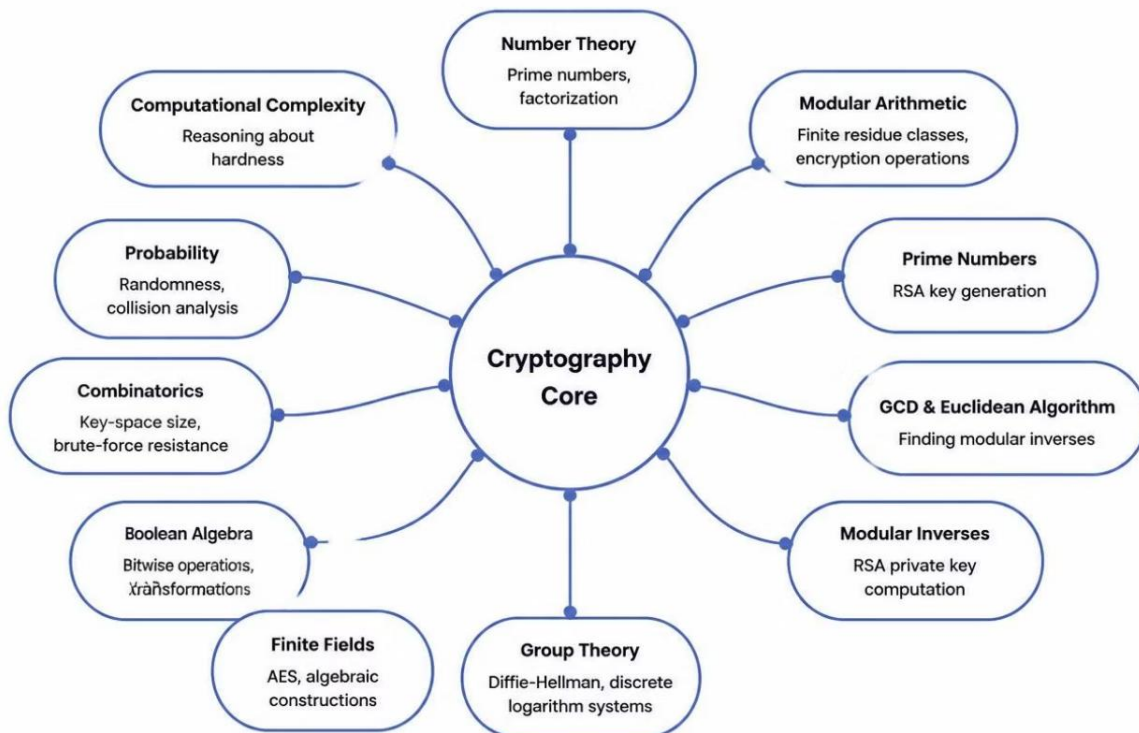


Figure. Mathematical toolkit connecting discrete mathematics concepts to cryptography.

3. MODULAR ARITHMETIC

Clock arithmetic

Modular arithmetic is often described as clock arithmetic because values wrap around after reaching a chosen modulus. On a 12-hour clock, five hours after 10 o'clock is 3 o'clock. In the same way, $15 \bmod 12$ equals 3.

A general example from the source is $17 \bmod 5 = 2$. This can also be written as $17 \equiv 2 \pmod{5}$. The congruence notation states that the two numbers have the same remainder when divided by the modulus.

$$17 \bmod 5 = 2$$

$$17 \equiv 2 \pmod{5}$$

Why it matters

Cryptographic algorithms repeatedly perform operations under a modulus. This creates a finite or controlled arithmetic environment and makes operations such as modular exponentiation practical to implement.

The important conceptual shift is that ordinary arithmetic is not the only way to think about numbers. In modular arithmetic, two different integers can represent the same residue class under a particular modulus.

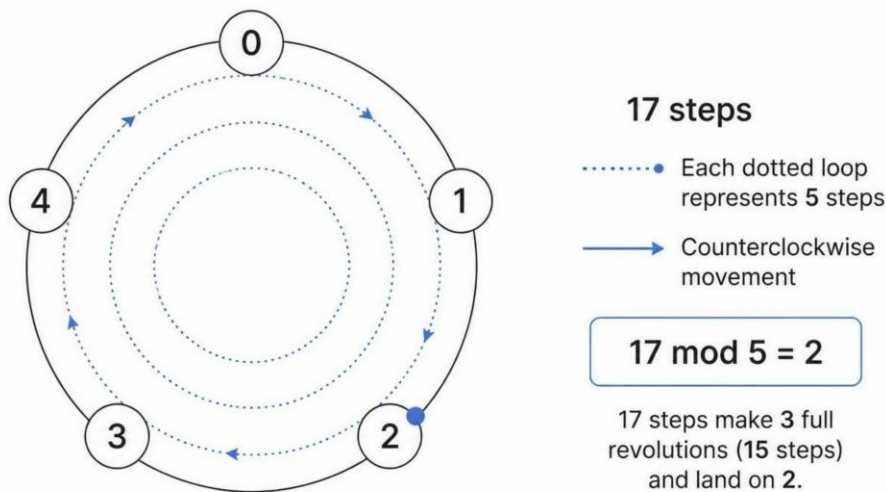


Figure. Modular arithmetic illustrated as clock-style wrap-around.

4. GCD, EUCLIDEAN ALGORITHM AND MODULAR INVERSE

GCD and Euclidean algorithm

The greatest common divisor, $\gcd(a,b)$, is the largest positive integer that divides both a and b . The Euclidean algorithm computes this efficiently by repeatedly replacing a pair of integers with the smaller integer and the remainder.

The extended Euclidean algorithm goes one step further. It can express the GCD as a linear combination of the original integers, which makes it useful for finding modular inverses.

Modular inverse

A modular inverse of a modulo n is a value a^{-1} satisfying $a \times a^{-1} \equiv 1 \pmod{n}$, when such an inverse exists.

The source example is $3 \times 27 \equiv 1 \pmod{40}$, because 81 leaves remainder 1 after division by 40. Therefore 27 is the modular inverse of 3 modulo 40.

This concept becomes essential in RSA because the private exponent is obtained as a modular inverse of the public exponent with respect to Euler's totient value.

$$a \times a^{-1} \equiv 1 \pmod{n}$$

$$3 \times 27 \equiv 1 \pmod{40}$$

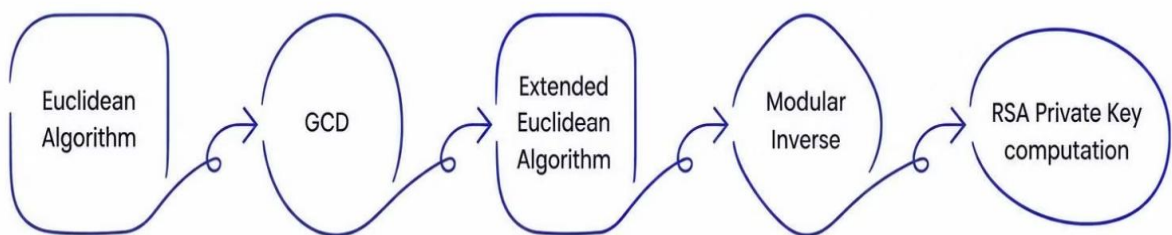


Figure. Euclidean algorithm → GCD → extended Euclidean algorithm → modular inverse → RSA.

5. PRIME NUMBERS AND EULER'S TOTIENT FUNCTION

Prime numbers

A prime number is an integer greater than 1 with exactly two positive divisors: 1 and itself. Prime numbers are fundamental building blocks in number theory because composite integers can be represented through prime factors.

RSA selects two distinct prime numbers p and q and multiplies them to form the modulus n . The source diagram shows the two primes feeding into $n = pq$.

Euler's totient function

Euler's totient function $\varphi(n)$ counts the positive integers up to n that are relatively prime to n . For two distinct primes p and q , the source uses $\varphi(n) = (p-1)(q-1)$ when $n = pq$.

This value is important for RSA because the public and private exponents are related through modular arithmetic with respect to $\varphi(n)$.

$$n = pq$$

$$\varphi(n) = (p - 1)(q - 1)$$

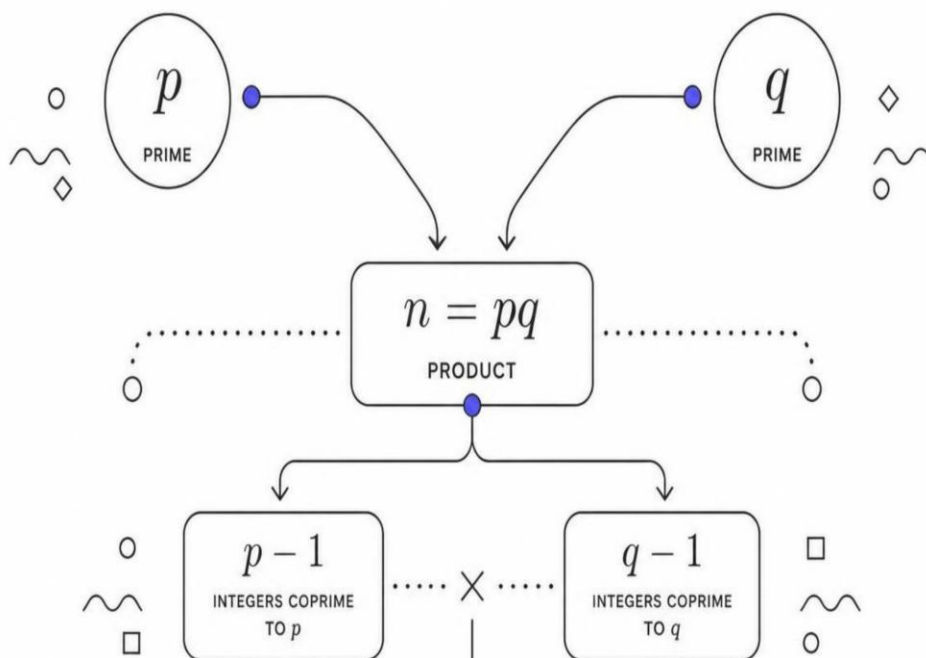


Figure. Prime numbers feeding into the RSA modulus and totient calculation.

6. RSA — DISCRETE MATHEMATICS IN ACTION

RSA key generation

RSA brings several discrete-mathematical ideas together in one public-key cryptosystem. The process begins by selecting two distinct prime numbers p and q .

The modulus is calculated as $n = pq$. Next, $\phi(n)$ is computed. A public exponent e is selected so that it is relatively prime to $\phi(n)$. The private exponent d is then chosen so that $ed \equiv 1 \pmod{\phi(n)}$.

The public key is the pair (e, n) , while the private key is the pair (d, n) . The source presentation describes this seven-step process as the point where the mathematical foundations culminate in cryptographic key generation.

$$ed \equiv 1 \pmod{\phi(n)}$$

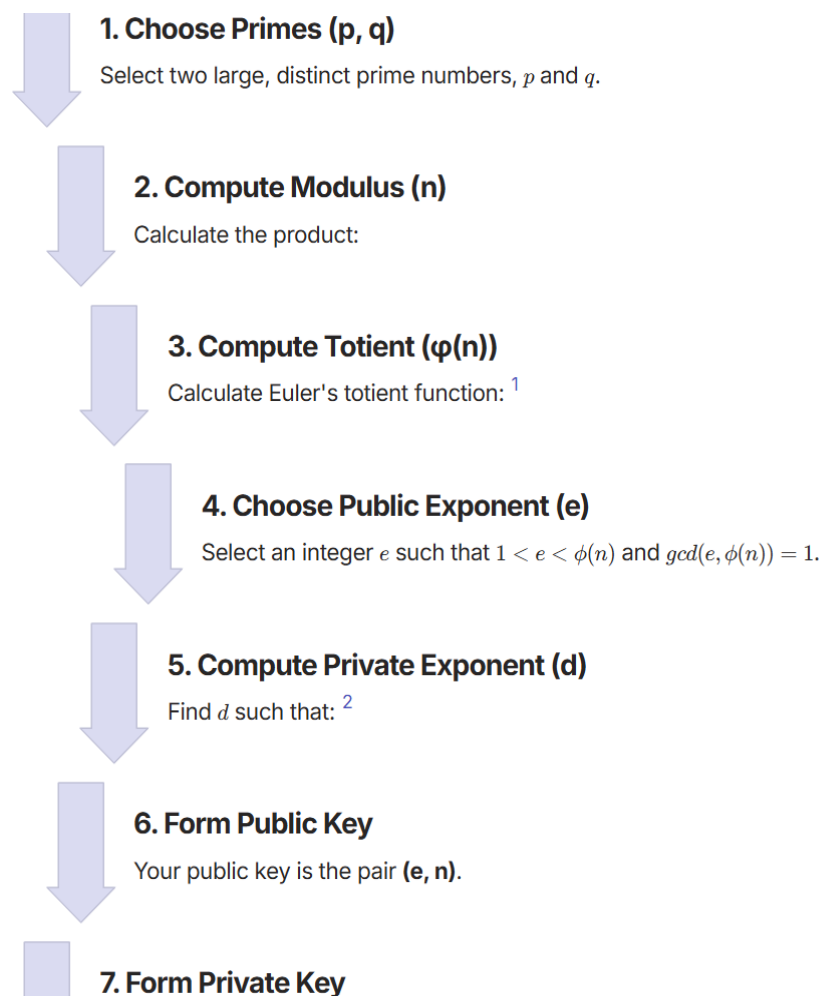


Figure. Seven-step RSA key-generation workflow.

7. RSA WORKED EXAMPLE

Key generation

To make RSA concrete, the source uses the illustrative values $p = 5$ and $q = 11$. Therefore $n = 5 \times 11 = 55$.

Euler's totient is $\varphi(55) = (5-1)(11-1) = 40$. We choose $e = 3$ because $\gcd(3,40) = 1$. The private exponent is $d = 27$ because $3 \times 27 = 81$ and $81 \equiv 1 \pmod{40}$.

Thus the public key is $(3,55)$ and the private key is $(27,55)$.

$$p = 5, q = 11, n = 55, \varphi(n) = 40, e = 3, d = 27$$

Encryption and decryption

For message $m = 7$, the source applies the RSA encryption relationship $c \equiv m^e \pmod{n}$. Therefore $c \equiv 7^3 \pmod{55} = 343 \pmod{55} = 13$.

The ciphertext is therefore 13. Decryption applies $m \equiv c^d \pmod{n}$, and modular exponentiation recovers the original message 7.

This small example is not intended to provide real-world security; it is a transparent numerical demonstration of how the mathematical relationships operate.

$$c \equiv m^e \pmod{n} \rightarrow 7^3 \pmod{55} = 13$$

$$m \equiv c^d \pmod{n} \rightarrow 13^{27} \pmod{55} = 7$$

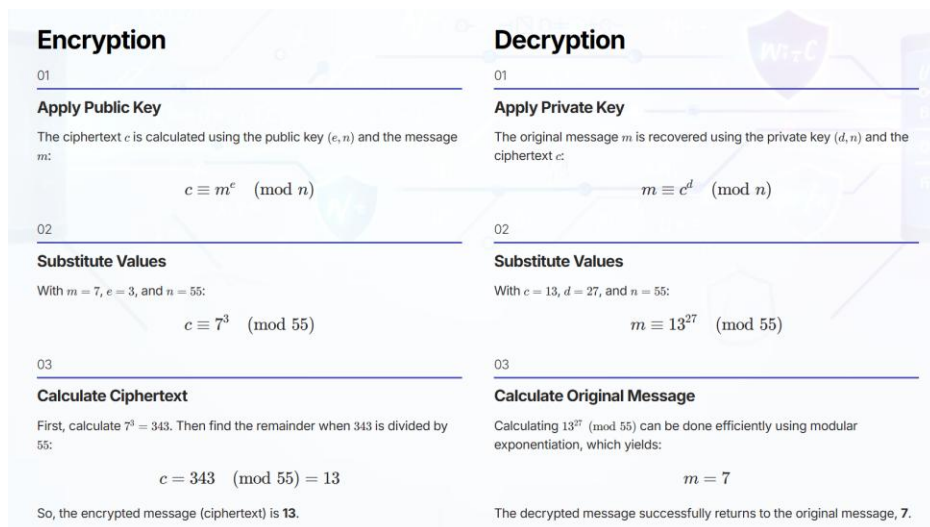


Figure. Worked RSA encryption/decryption example from the supplied project material.

8. WHY RSA IS SECURE: THE MATHEMATICAL TRAPDOOR

Easy forward operation, difficult reverse problem

The source project describes RSA security through a mathematical asymmetry. Multiplying two primes is straightforward. Given only their product, however, recovering the original large prime factors can be computationally difficult when appropriate parameters are used.

This is the trapdoor intuition: the public information allows legitimate operations, while the secret structure represented by the prime factors enables the corresponding private operation. The important term is computational difficulty rather than mathematical impossibility.

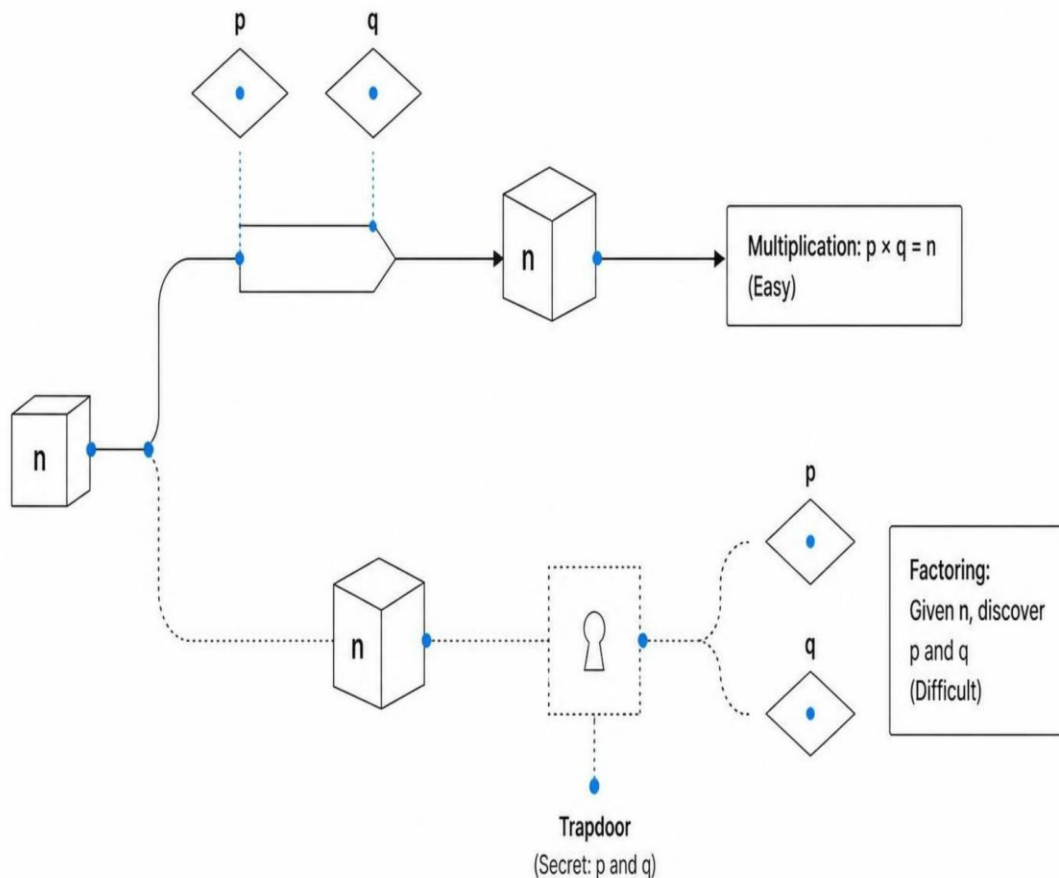


Figure. RSA trapdoor intuition: easy multiplication versus difficult factorization.

9. DIFFIE-HELLMAN KEY EXCHANGE

Creating a shared secret

Diffie-Hellman addresses a different problem: two parties want to establish a shared secret over a public channel. Alice chooses a private value a and publishes $A = g^a$. Bob chooses a private value b and publishes $B = g^b$.

Alice computes $B^a = g^{ab}$, while Bob computes $A^b = g^{ab}$. Both therefore obtain the same shared value.

The security idea is connected to the difficulty of the discrete logarithm problem in suitable mathematical groups. The source presentation emphasizes the equality of the two independently computed shared values.

$$B^a = (g^b)^a = g^{ab} \quad \text{and} \quad A^b = (g^a)^b = g^{ab}$$

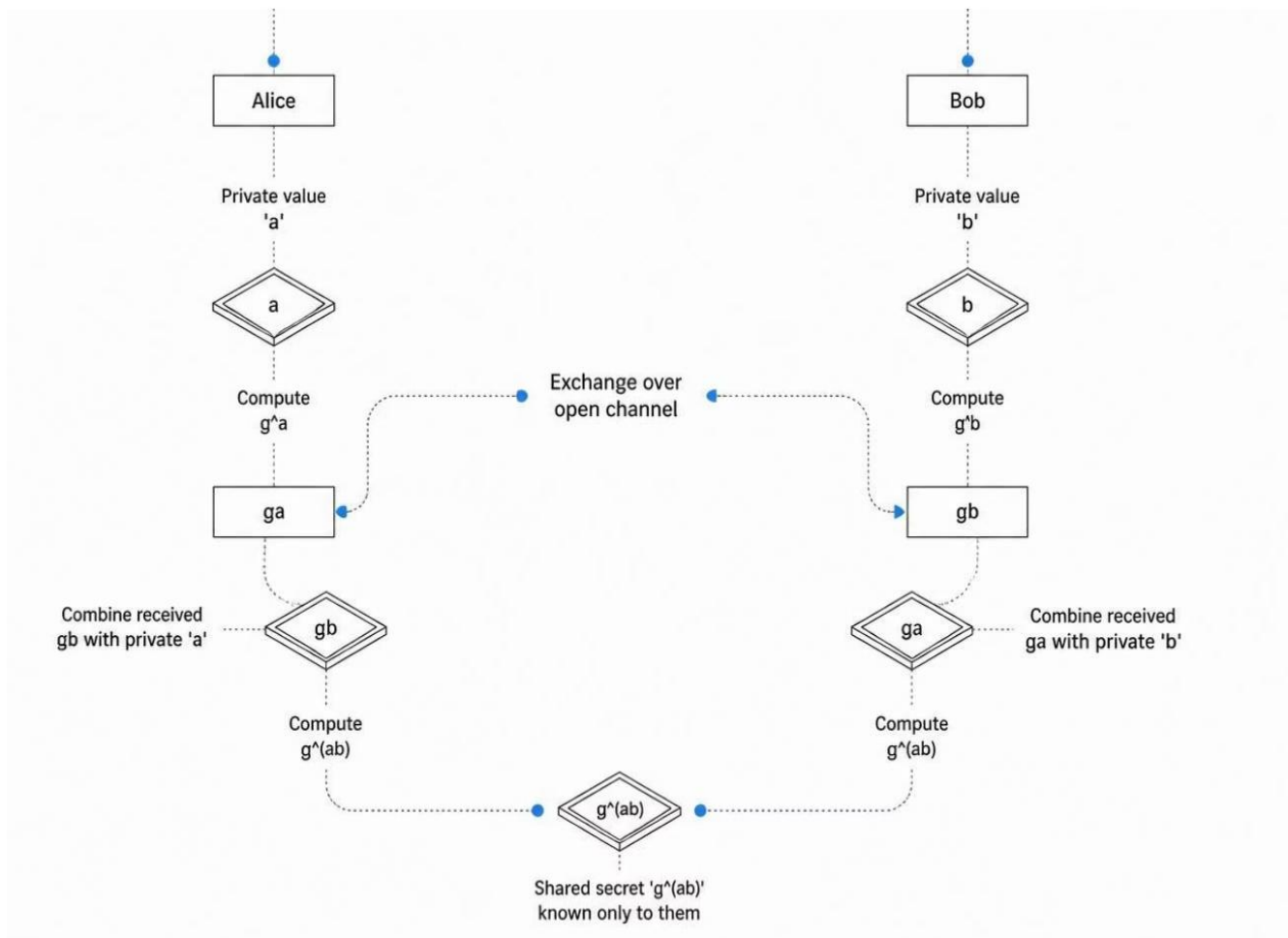


Figure. Diffie-Hellman shared-secret construction.

10. GROUPS AND FINITE FIELDS

Algebraic structures

A group is a mathematical structure equipped with an operation satisfying closure, associativity, identity and inverse properties. A cyclic group is a group in which the elements can be generated from a single generator.

A finite field is a finite algebraic system in which addition, subtraction, multiplication and division are defined consistently for the field elements. These structures provide controlled mathematical environments for cryptographic algorithms.

Cryptographic connections

The source connects cyclic groups to Diffie–Hellman, elliptic curves over finite fields to elliptic-curve cryptography, and finite-field arithmetic to AES. The larger lesson is that abstract algebra is not merely theoretical; its properties determine which cryptographic operations are possible and how their security assumptions are formulated.

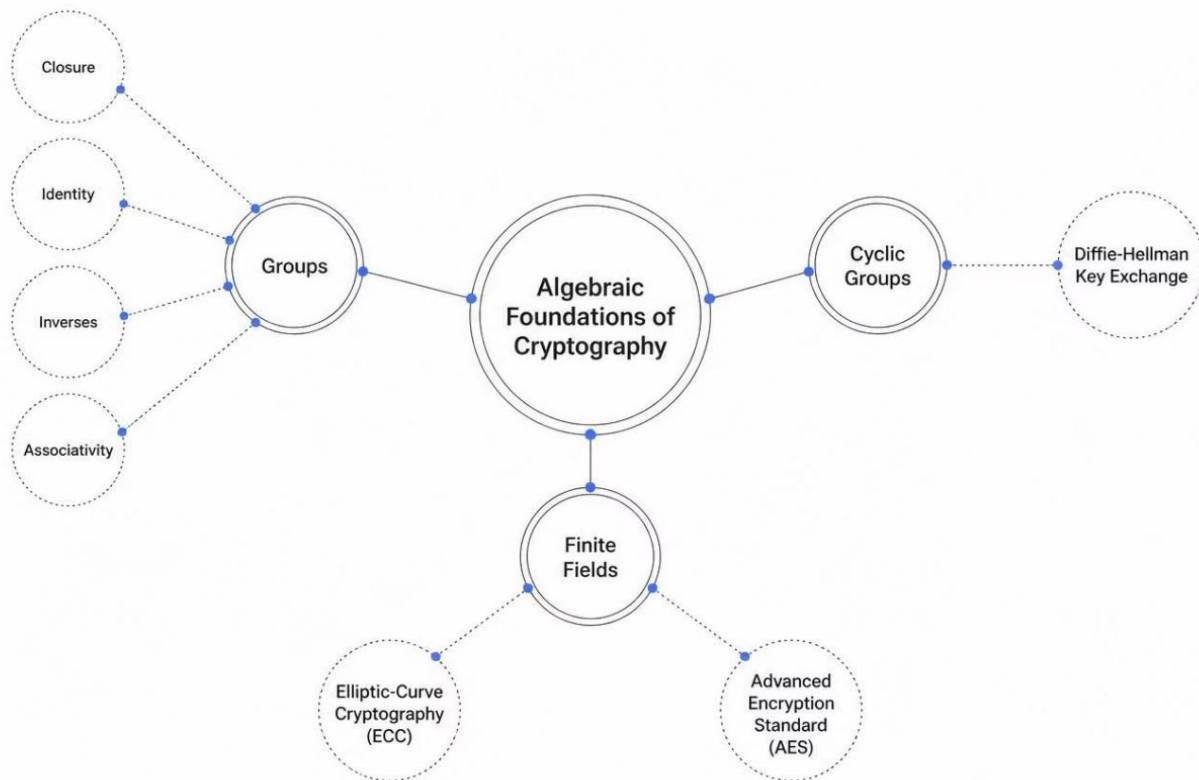


Figure. Algebraic foundations connecting groups, cyclic groups and finite fields to cryptography.

1 1. BOOLEAN ALGEBRA AND AES

Boolean operations

Boolean logic is fundamental to digital computation. The basic operations highlighted by the source are AND, OR, XOR and NOT. These operations work directly with binary values and therefore connect naturally to computer hardware and digital data.

Finite fields inside AES

The source explains that AES combines Boolean operations with finite-field arithmetic. AES operates on bytes and uses operations over $GF(2^8)$.

A simplified AES pipeline is presented as Input $\rightarrow$ SubBytes $\rightarrow$ ShiftRows $\rightarrow$ MixColumns $\rightarrow$ AddRoundKey $\rightarrow$ Ciphertext. SubBytes uses a non-linear S-box involving finite-field inversion; ShiftRows permutes the state; MixColumns performs finite-field polynomial multiplication; and AddRoundKey combines the state with a round key using XOR.

This is a strong example of discrete mathematics becoming a concrete symmetric encryption algorithm.

$$GF(2^8)$$

Simplified AES Pipeline



Input

Original plaintext data enters the encryption process.



ShiftRows

Rows of the state matrix are cyclically shifted to permute the data.



AddRoundKey

The current state is combined with a round key using bitwise XOR operations.



SubBytes

Each byte is replaced with another byte using a non-linear substitution table (S-box), involving finite field inversion.



MixColumns

Columns of the state matrix are mixed using polynomial multiplication over a finite field $GF(2^8)$, diffusing changes across bytes.



Ciphertext

The final encrypted output is produced after multiple rounds.

Figure. Simplified AES pipeline showing the role of finite-field and Boolean operations.

12. HASHING, COMBINATORICS AND PROBABILITY

Hash functions

Cryptography is broader than encryption. A hash function maps arbitrary-length input data to a fixed-length digest. Hashing is used for integrity checking, password-storage constructions, and as a component of digital-signature systems.

A central security requirement is collision resistance: it should be computationally difficult to find two different inputs that produce the same hash output.

Combinatorics and the birthday paradox

Combinatorics helps describe the size of possible input and output spaces. Probability helps us reason about the chance of accidental collisions.

The source connects this reasoning to the birthday paradox: collisions become plausible much sooner than a simple one-to-one intuition might suggest when many random inputs are considered.



Figure. Input data → hash function → fixed-length digest.

13. COMPUTATIONAL COMPLEXITY AND PRACTICAL SECURITY

Possible is not always practical

A problem can be mathematically solvable while still being computationally impractical. Cryptographic security often depends on making an attack require an unreasonable amount of computation, time or memory.

The source visualizes a progression from Problem → Algorithm → Computation → Time/Memory → Practical Security. This captures the central role of computational complexity in deciding whether an attack is realistic.

Security assumptions

Cryptographic strength is therefore not simply a statement that an algorithm cannot be broken. It is a statement about the assumptions, parameters and resources under which known attacks remain infeasible.

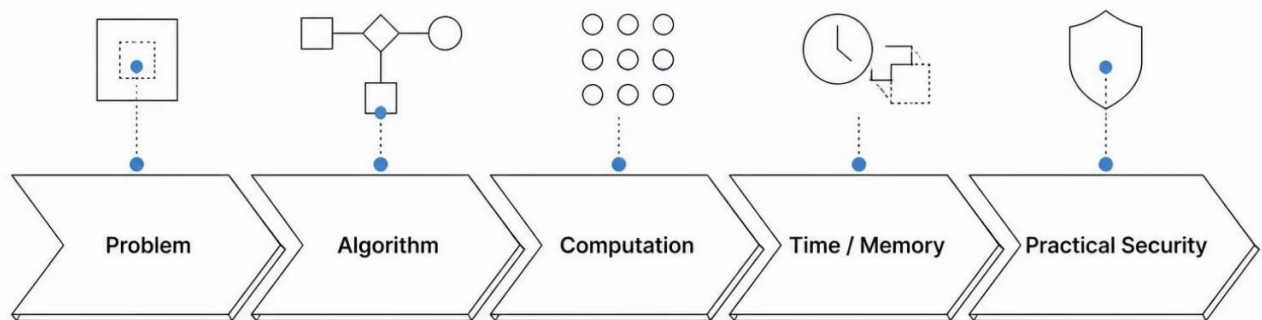
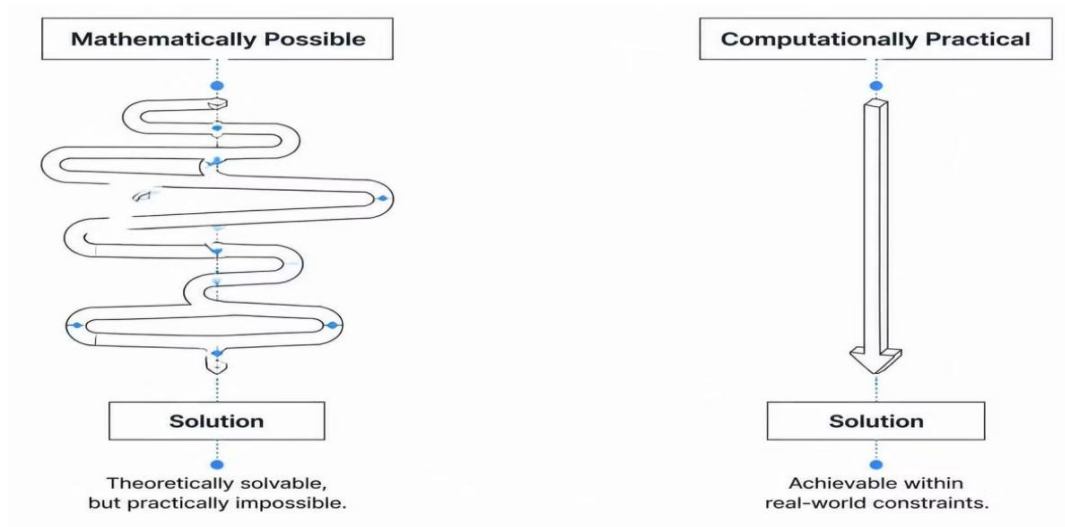


Figure. Problem → algorithm → computation → time/memory → practical security.



14. REAL-WORLD APPLICATIONS

Applications

The source maps mathematical ideas to several digital applications. HTTPS/TLS is associated with number theory, modular arithmetic, algebra, group theory and hashing. Secure messaging is associated with group-theoretic and finite-field constructions. Digital signatures combine asymmetric cryptography with hashing.

The source also identifies online banking and digital payments as applications involving combinations of asymmetric and symmetric cryptography, hashing, group theory and elliptic-curve methods.

The key message is that the mathematics is not isolated inside a classroom exercise. It becomes part of the machinery that supports secure digital communication and transactions.

MATHEMATICS	Number Theory, Modular Arithmetic, Algebra, Group Theory, Hashing
APPLICATION	Secure Messaging End-to-end encryption MATHEMATICS Group Theory (Diffie-Hellman, ECC), Finite Fields (AES)
APPLICATION	Digital Signatures Authentication + Integrity MATHEMATICS Number Theory, Modular Arithmetic, Algebra, Hashing
APPLICATION	Online Banking Secure transactions MATHEMATICS Asymmetric/Symmetric Cryptography (RSA, AES), Hashing, Group Theory
APPLICATION	Digital Payments Payment security MATHEMATICS Elliptic Curve Cryptography (ECC), Hashing, Number Theory

Figure. Applications mapped to the mathematical concepts used by cryptographic systems.

15. QUANTUM COMPUTING AND THE FUTURE

Changing mathematical assumptions

Cryptography is closely connected to the mathematical problems that are considered difficult to solve. A cryptographic system is secure only as long as the mathematical assumptions on which it is based remain computationally difficult for an attacker. This raises an important future-oriented question: if the computational model changes, can the mathematical foundations of cryptography also be challenged?

Classical public-key cryptographic systems such as RSA rely heavily on problems such as integer factorization, while other systems rely on the discrete logarithm problem. For conventional computers, solving these problems for sufficiently large parameters is computationally impractical. This computational difficulty forms an important part of the security foundation of modern public-key cryptography.

The development of quantum computing introduces a different computational model. Quantum computers use quantum-mechanical principles to process information and, for certain mathematical problems, can provide algorithms that are fundamentally different from their classical counterparts. In 1994, Peter Shor proposed a quantum algorithm that can efficiently solve integer factorization and discrete logarithm problems on a sufficiently powerful quantum computer. Consequently, a cryptographically relevant quantum computer could seriously threaten systems such as RSA and traditional discrete-logarithm-based cryptography.

Shor's Algorithm and the Threat to RSA

The connection with RSA can be understood from its mathematical foundation. RSA constructs a public modulus by multiplying two large prime numbers:

$$n = pq$$

Multiplication of p and q is computationally easy, but recovering the original prime factors from a sufficiently large n is considered difficult for classical computers. This asymmetry is an important part of RSA's security.

Shor's algorithm changes this assumption. A sufficiently powerful quantum computer could use the algorithm to factor large integers much more efficiently than known classical algorithms. Therefore, the mathematical problem that provides security in the classical setting would no longer provide the same level of protection in the quantum setting. NIST notes that sufficiently powerful quantum computers could break public-key systems based on factoring and discrete logarithms.

This does not mean that existing RSA systems are already broken. Practical, cryptographically relevant quantum computers capable of performing such attacks at the required scale do not currently exist, and the precise timeline for developing them is uncertain. However, the potential impact is significant enough that researchers and standards organizations are preparing for this possibility now.

Post-Quantum Cryptography

The possible future threat from quantum computers has led to the development of Post-Quantum Cryptography (PQC). The objective of PQC is to design cryptographic algorithms that can remain secure against attacks from both classical and quantum computers.

Instead of depending primarily on integer factorization or conventional discrete logarithms, post-quantum algorithms investigate different mathematical problems. Major approaches include structured lattice-based cryptography and hash-based cryptography, among others. NIST has developed standards for post-quantum key establishment and digital signatures, including FIPS 203, FIPS 204, and FIPS 205, which were finalized in August 2024.

This development provides a direct connection with the main theme of this project. It demonstrates that cryptography does not depend on one permanent mathematical problem. Instead, cryptographers continually search for mathematical structures that provide useful security properties and remain computationally difficult for available forms of computation.

The Broader Mathematical Lesson

The quantum-computing challenge demonstrates an important principle:

Cryptographic Security $\neq$ Mathematics Alone

Rather, security depends on the combination of:

Mathematical Problem + Computational Model + Algorithmic Assumptions
--

A problem that is difficult for a classical computer may become easier under a different computational model. Therefore, cryptographic systems must be evaluated not only according to their mathematical structure but also according to the capabilities of current and future computing technologies.

The transition toward post-quantum cryptography also illustrates the importance of cryptographic agility. Large computer systems cannot replace cryptographic algorithms instantly because algorithms are deeply integrated into operating systems, websites, financial infrastructure, communication protocols, hardware, and software. NIST therefore emphasizes

the need to begin preparing for the transition before large-scale quantum computers become available.

Another concern is the possibility of “harvest now, decrypt later.” An attacker could collect encrypted information today and store it with the expectation that a sufficiently powerful quantum computer may be able to decrypt some of that information in the future. This is particularly relevant for information that must remain confidential for many years.

Future of Cryptography

The future of cryptography will therefore involve a continuous interaction between mathematics, computer science, and emerging technology. New computational models can challenge existing assumptions, while new mathematical problems can provide foundations for future security systems.

The central lesson is that cryptography is an evolving field. The mathematical principles of modular arithmetic, number theory, algebra, probability, and computational complexity remain fundamental, but the specific mathematical problems selected for security may change as computational capabilities develop.

Thus, quantum computing does not make mathematics less important; rather, it makes the choice and analysis of mathematical foundations even more important. The transition from classical cryptography toward post-quantum cryptography is an example of how mathematical theory continuously adapts to technological change.

Classical Mathematics → Classical Cryptography → Quantum Challenge → Post-Quantum Mathematics → Future Cryptography

16. CONCLUSION

The central idea

Discrete mathematics provides the structures and reasoning tools from which cryptographic algorithms are constructed. Modular arithmetic gives controlled numerical operations; number theory supplies primes and factorization; GCD and modular inverses support RSA key generation; groups support key exchange; finite fields support AES and elliptic-curve constructions; and probability and complexity help evaluate security.

The RSA example demonstrates the complete journey from simple mathematical quantities to a functioning encryption and decryption process. The other examples show that the same mathematical mindset extends beyond RSA into key exchange, symmetric encryption, hashing and future cryptographic design.

Cryptography is therefore not simply about hiding information. It is about constructing security from mathematical structure.

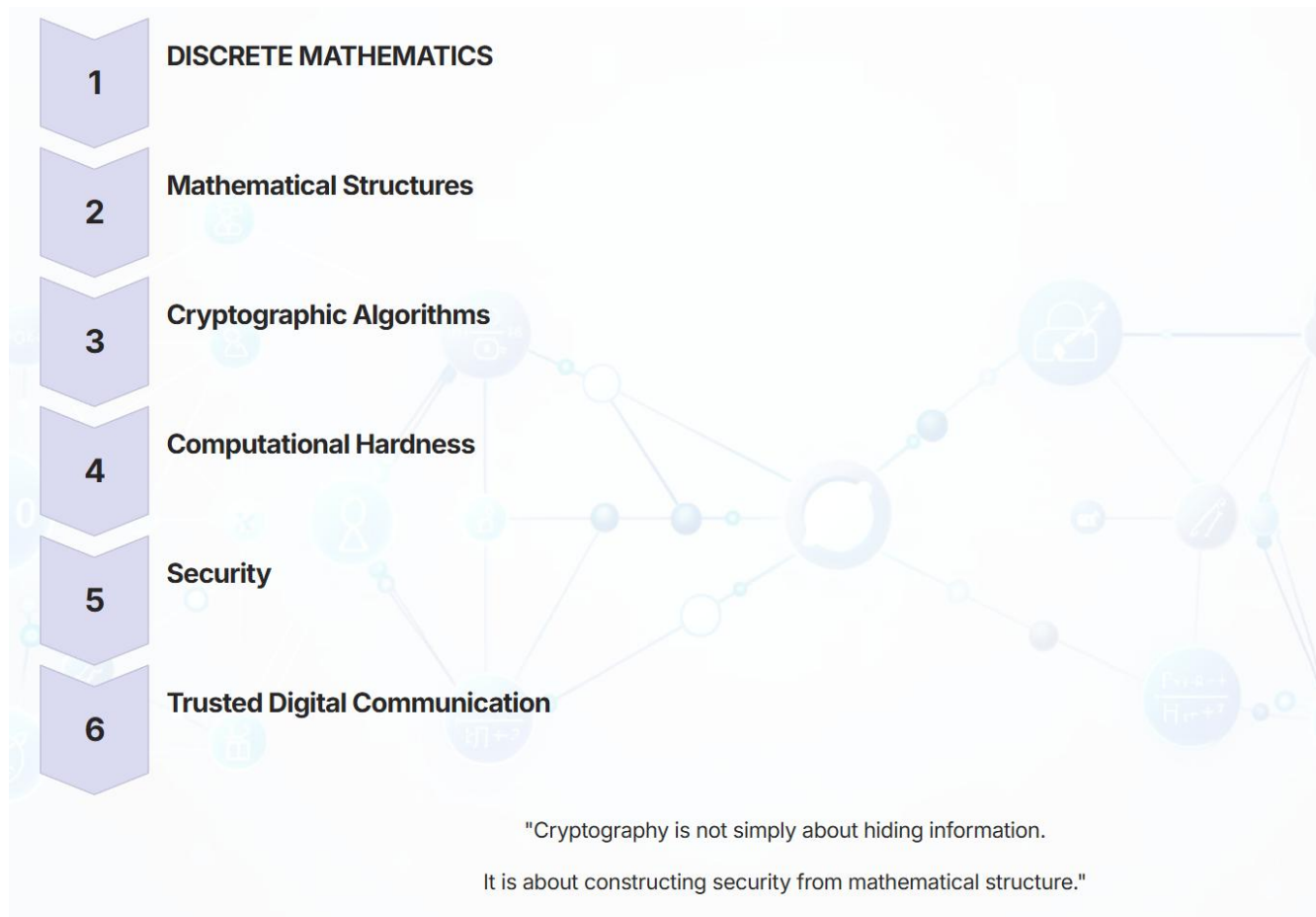


Figure. The big picture: discrete mathematics leading to trusted digital communication.

REFERENCES

The following references are taken from the supplied project material and are retained as the primary bibliographic base for this report.

1. MIT OpenCourseWare, Mathematics for Computer Science — course materials on discrete mathematics, modular arithmetic and cryptography.
2. MIT OpenCourseWare, Principles of Discrete Applied Mathematics — materials on modular arithmetic, RSA, cryptography and primality testing.
3. Menezes, A. J., van Oorschot, P. C., & Vanstone, S. A., Handbook of Applied Cryptography, CRC Press.
4. MIT OpenCourseWare, Lecture 15: Modular Arithmetic and Elementary Algebra, Spring 2024.
5. MIT OpenCourseWare, Lecture 10: Modular Arithmetic, covering modular inverses, Euclidean algorithm and CRT.
6. MIT CSAIL, Review of Modular Arithmetic, 6.857.
7. MIT OpenCourseWare, Cryptography Notes: The RSA Code.
8. National Institute of Standards and Technology (NIST), Advanced Encryption Standard (AES).
9. Stallings, William, Cryptography and Network Security: Principles and Practice, Pearson.
10. Rosen, Kenneth H., Discrete Mathematics and Its Applications, McGraw-Hill.
11. Katz, Jonathan & Lindell, Yehuda, Introduction to Modern Cryptography, CRC Press.

Note: The diagrams in this report are reproduced/adapted from the supplied project PDF so that the written report remains visually consistent with the presentation material.

FINAL TAKEAWAY

*“Cryptography is not simply about hiding information.
It is about constructing security from mathematical structure.”*